

Campus Placement Management System (CPMS) - Project Report

Name: Abhishek Kumar

Roll No: DS24F2004690

Email: 24F2004690@ds.study.iitm.ac.in

A full-stack, asynchronous web application designed to streamline the campus placement process for Students, Recruiters, and University Admins.

Built with a Vue 3 frontend and a Flask backend, this project leverages Redis and Celery for advanced caching, background job processing, and automated email scheduling.

Key Features:

- **Role-Based Access Control (RBAC):** Distinct dashboards and permissions for Admins, Companies, and Students using JWT Authentication.
- **Fast API:** Heavy database queries (like student directories) are cached in memory using Redis, dropping response times from ~50ms to ~2ms.
- **Asynchronous Tasks:** CSV data exports are handed off to a Celery background worker, allowing the UI to remain responsive while polling for the download link.
- **Automated Email Reminders:** A Celery Beat scheduler automatically emails students about placement drives closing within 24 hours.
- **Automated Reporting:** Generates and emails a beautifully formatted HTML activity report to Admins on the 1st of every month.
- **Session Security:** Custom global fetch interceptors and JWT lookup loaders ensure deleted users or expired sessions are instantly and securely logged out.

Tech Stack:

Frontend:

- Vue 3 (Composition API) & Vue Router
- Bootstrap 5 & Bootstrap Icons
- Chart.js & vue-chartjs (Data Visualization)

Backend:

- Python 3 / Flask
- Flask-SQLAlchemy (ORM) & SQLite
- Flask-JWT-Extended (Authentication)
- Flask-Caching & Redis (Cache & Message Broker)
- Flask-Mail (SMTP Emails)
- Celery & Celery Beat (Background Jobs & Cron Scheduling)

API Endpoint Reference

summary of all REST API endpoints, grouped by module.

Authentication

Method	Endpoint	Role	Description
GET	/	Public	Basic health check to confirm the API is running.
POST	/api/login	Public	Authenticates a user and returns a JWT access token.
POST	/api/register/student	Public	Registers a new student account.
POST	/api/register/company	Public	Registers a new company account, pending Admin approval.
GET	/api/verify_session	Any (JWT)	Verifies the current JWT session is still valid and the account is not blacklisted.

Admin

Method	Endpoint	Role	Description
GET	/api/admin/global_search	Admin	Searches students, companies, and drives from one query.
GET	/api/admin/stats	Admin	Returns dashboard summary counts (students, companies, drives).
GET	/api/admin/companies	Admin	Lists all registered companies with approval/blacklist status.
PUT	/api/admin/approve_company/{id}	Admin	Approves a pending company registration.
PUT	/api/admin/blacklist_company/{id}	Admin	Toggles a company's blacklist status.
GET	/api/admin/drives	Admin	Lists all placement drives across every company.
PUT	/api/admin/approve_drive/{id}	Admin	Approves a pending placement drive.
PUT	/api/admin/reject_drive/{id}	Admin	Rejects a pending placement drive.
GET	/api/admin/students	Admin	Lists all registered students (response cached 60s).
PUT	/api/admin/blacklist_student/{id}	Admin	Toggles a student's blacklist status.
GET	/api/admin/analytics	Admin	Returns placement analytics: application funnel, branch-wise selections, and drive trends.

Recruiter (Company)

Method	Endpoint	Role	Description
GET	/api/recruiter/global_search	Company	Searches the company's own drives and their applicants.
GET	/api/recruiter/drives	Company	Lists placement drives posted by the logged-in company.
POST	/api/recruiter/drives	Company	Posts a new placement drive (starts as Pending, needs Admin approval).
PUT	/api/recruiter/drives/{id}	Company	Updates a drive owned by the logged-in company.
DELETE	/api/recruiter/drives/{id}	Company	Deletes an owned drive, if it has no active applications.
GET	/api/recruiter/applications	Company	Lists all applications received across the company's drives.
PUT	/api/recruiter/applications/{id}	Company	Updates an applicant's status, remarks, and interview schedule.

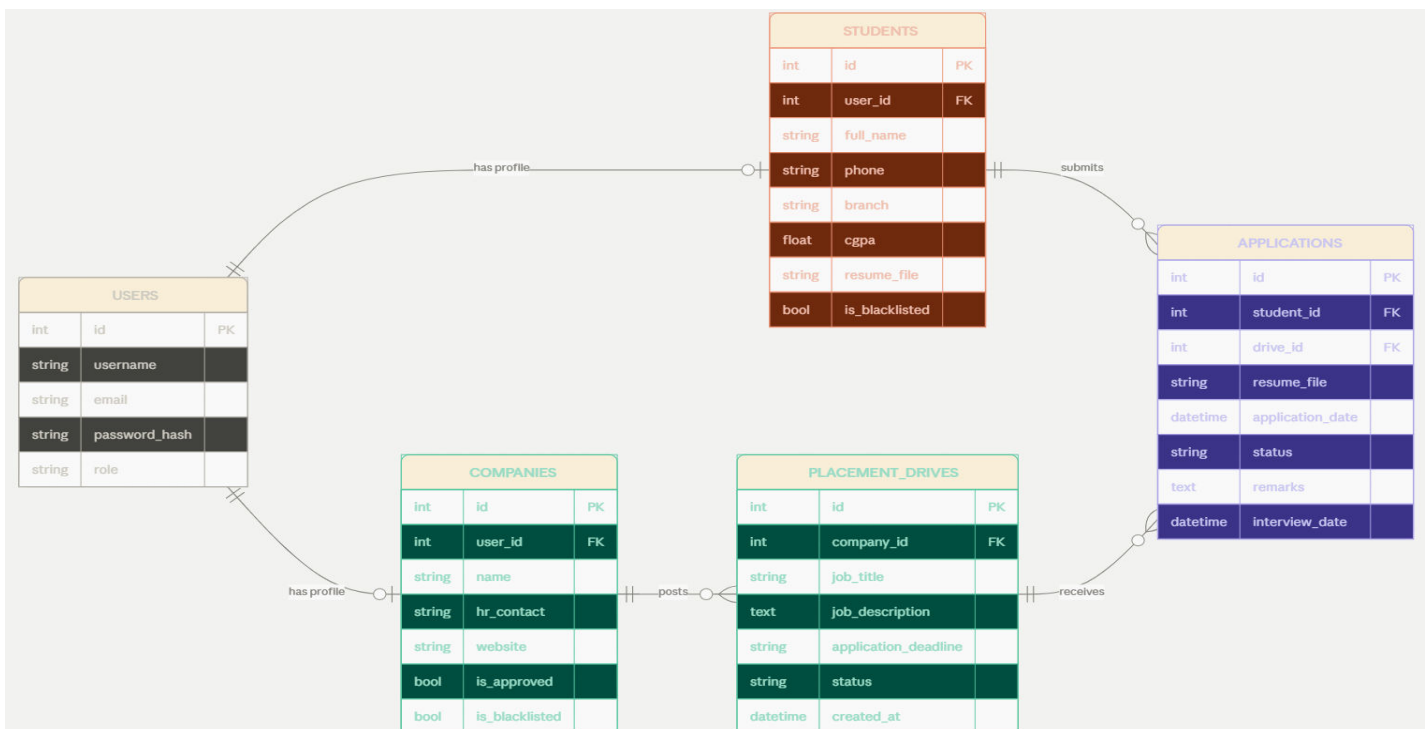
Student

Method	Endpoint	Role	Description
GET	/api/student/global_search	Student	Searches open drives and the student's own applications.
GET	/api/student/profile	Student	Retrieves the logged-in student's profile.
PUT	/api/student/profile	Student	Updates the logged-in student's profile details.
POST	/api/student/upload_resume	Student	Uploads the student's resume as a PDF file.
GET	/api/student/drives	Student	Lists open, Admin-approved drives with an applied/not-applied flag (cached per-user, 120s).
POST	/api/student/apply	Student	Submits an application to a placement drive.
GET	/api/student/applications	Student	Lists all applications submitted by the logged-in student.
POST	/api/student/export_applications	Student	Queues an async task to export the student's applications as CSV.

Tasks & Files

Method	Endpoint	Role	Description
GET	/api/tasks/{task_id}	Any (JWT)	Polls the status/result of a queued async task (e.g. CSV export).
GET	/uploads/{filename}	Public	Serves an uploaded file, such as a student's resume.

Database Schema/ER diagram:



Relationships:

- **Users → Companies:** One-to-one — a user with role company has exactly one company profile.
- **Users → Students:** One-to-one — a user with role student has exactly one student profile.
- **Companies → Placement_drives:** One-to-many — a company can post multiple placement drives.
- **Students → Applications:** One-to-many — a student can submit multiple applications.
- **Placement_drives → Applications:** One-to-many — a drive can receive applications from multiple students.

LLM Declaration:

I had some help with gemini in suggesting frontend code including design ideas, api suggestions in backend and document formatting, the limit to which is around 20-25%.

Prerequisites:

Node.js (v16 or later)

Python (v3.8 or later)

WSL (Ubuntu) on Windows, or a native Linux/macOS environment

Redis Server inside the WSL/Linux environment

Using the app:

1. Backend:

Navigate to the backend folder and activate the virtual environment **python -m venv venv**

Activate it using **venv\Scripts\activate - - - (on windows)**

source venv/bin/activate - - - (on mac/linux)

Install the required dependencies using **pip install -r requirements.txt**

Init the database and create admin - - - **python init_db.py** (change admin credentials accordingly)

Change the following in app.py to enable E-mail functionality:

```
MAIL_USERNAME=your_gmail@gmail.com
MAIL_PASSWORD=your_16_char_google_app_password
JWT_SECRET_KEY=your_super_secret_jwt_key
```

Start the server - - - **python app.py**

2. Frontend:

install the packages - - - **npm install**

npm run dev the app will be available at - - - **http://localhost:5173**

3. Start Redis and Celery Services (WSL/Linux):

Navigate to the project folder - - - **cd /mnt/c/path/to/your/cpms/backend**

Start the Redis server - - - **sudo service redis-server start**

Create and activate a Linux virtual environment - - - **python3 -m venv venv_wsl**
source venv_wsl/bin/activate

Install the dependencies - - - **pip install -r requirements.txt**

Start the Celery worker and scheduler - - - **celery -A app.celery worker -B --loglevel=info**

Drive link for video demo:

https://drive.google.com/file/d/1bqMsSRcGBMUyjuvDoTssU33ImznqQ_yks/view?usp=sharing